Boletíon ejercicios repaso

Ejercicio 1: Servidor web con Nginx

Crea una configuración Docker que cumpla lo siguiente:

- Construir una imagen personalizada basada en Ubuntu
- Instalar el servicio Nginx
- Configurar el contenedor para que:
  - Arranque automáticamente el servicio al iniciarse
  - Exponga el puerto 80
- Crear un archivo docker-compose.yml que levante el servicio
- (Opcional) Añadir una página HTML personalizada

Solución

Docker file

dockerfile

```
FROM ubuntu:22.04

# Evita interacción en instalación
ENV DEBIAN_FRONTEND=noninteractive

# Actualizar e instalar nginx
RUN apt-get update && \
    apt-get install -y nginx && \
    apt-get clean

# Copiar página personalizada (opcional)
COPY index.html /var/www/html/index.html

# Exponer puerto 80
EXPOSE 80

# Arrancar nginx en primer plano
CMD ["nginx", "-g", "daemon off;"]
```

Docker Compose yml

YAML

```yaml
version: '3.8'

services:
  web:
    build: .
    container_name: nginx_server
    ports:
      - "8080:80"
    restart: always
```

Index.html (opcional)

HTML

```html
<!DOCTYPE html>
<html>
<head>
    <title>Mi servidor Docker</title>
</head>
<body>
    <h1>Servidor Nginx funcionando en Docker</h1>
</body>
</html>
```

Ejercicio 2: Servidor SSH en contenedor

Crea una configuración Docker que cumpla lo siguiente:

- Construir una imagen basada en Ubuntu
- Instalar el servicio OpenSSH Server
- Crear un usuario con contraseña
- Configurar el servicio SSH para que:
  - Se inicie automáticamente al arrancar el contenedor
  - Escuche en el puerto 22
- Crear un docker-compose.yml que permita conectarse desde el host al contenedor

Dockerfile

```
ENV DEBIAN_FRONTEND=noninteractive

# Instalar openssh-server
RUN apt-get update && \
    apt-get install -y openssh-server sudo && \
    mkdir /var/run/sshd

# Crear usuario
RUN useradd -m alumno && \
    echo "alumno:alumno123" | chpasswd && \
    adduser alumno sudo

# Permitir login por contraseña
RUN sed -i 's/#PasswordAuthentication yes/PasswordAuthentication yes/' /etc/ssh/sshd_config

# Exponer puerto 22
EXPOSE 22

# Arrancar sshd
CMD ["/usr/sbin/sshd", "-D"]
```

Solución

Docker-compose.yml

```yaml
version: '3.8'

services:
  ssh_server:
    build: .
    container_name: ssh_container
    ports:
      - "2222:22"
    restart: always
```

Cómo probarlo

- Ejercicio 1:
  - Abrir navegador en: `http://localhost:8080`
- Ejercicio 2:

```bash
ssh alumno@localhost -p 2222
```

Contraseña: `alumno123`

Ejercicio 3

Se ha proporcionado a los alumnos el siguiente archivo docker-compose.yml, pero contiene varios errores que impiden que los servicios funcionen correctamente.

```yaml
version: '3.9'

services:
  web:
    image: nginx
    container_name: web_server
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html
    depends_on:
      - dbs

  app:
    build: .
    container_name: app_server
    ports:
      - "3000-3000"
    environment:
      - DATABASE_URL=mysql://root:root@db:3306/app_db
    depends_on:
      - db
```

```yaml
   db:
     image: mysql:8
     container_name: database
     restart: always
     environment:
       MYSQL_ROOT_PASSWORD: root
       MYSQL_DATABASE: app_db
     ports:
       - "3306:3306"
     volumes:
       - db_data:/var/lib/mysql

 volumes:
   dbdata:
```

- Identificar al menos 5 errores en el archivo.
- Explicar por qué son errores.
- Corregir el archivo para que funcione correctamente.
- (Opcional) Ejecutar docker-compose up y comprobar que:
    - Nginx sirve contenido
    - La app se conecta a la base de datos

Solución

➔ depends_on: - dbs
- Servicio inexistente (dbs)
- Debe ser db

➔ Puerto mal definido
- Sintaxis incorrecta
- Debe ser "3000:3000"

➔ Nombre del volumen inconsistente

volumes:

- db_data:/var/lib/mysql

- Se usa db_data pero se declara dbdata
- Deben coincidir

➔ Posible problema en DATABASE_URL
- El hostname debe coincidir con el servicio (db)
- En este caso está bien, pero es un punto clave a revisar

➔ Falta versión explícita en imagen nginx
- No es error crítico, pero mala práctica usar latest implícito

Ejercicio 4

Dispones de un archivo HTML básico. Tu tarea es:

- Crear una imagen Docker que sirva ese archivo usando Nginx.
- Levantar un contenedor a partir de esa imagen.
- Comprobar que la web funciona en el navegador.

```html
<!DOCTYPE html>
<html>
<head>
  <title>Mi Web Docker</title>
</head>
<body>
  <h1>Hola desde Docker</h1>
  <p>Este es un ejercicio de DAW</p>
</body>
</html>
```

Solución

1) Crear un archivo Dockerfile que:
   - Use una imagen base de Nginx
   - Copie el archivo index.html al directorio adecuado
2) Construir la imagen con:
   - docker build -t mi-web .
3) Ejecutar el contenedor:
   - docker run -d -p 8080:80 mi-web
4) Acceder desde el navegador a:
   - docker run -d -p 8080:80 mi-web

Docker correcto y completo:

```
FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html
```

Ejercicio 5

Se quiere desplegar una aplicación formada por:

- Un servidor web con **Nginx**
- Una base de datos **MySQL**

Tu tarea es crear un archivo docker-compose.yml que:

1. Levante ambos servicios
2. Exponga el servidor web en el puerto 8080
3. Configure la base de datos con:
    - usuario: `user`
    - contraseña: `password`
    - base de datos: `mi_app`
4. Cree un volumen para persistir los datos de MySQL

Solución

```yaml
version: '3.9'

services:
  web:
    image: nginx:latest
    container_name: web_server
    ports:
      - "8080:80"
    volumes:
      - ./html:/usr/share/nginx/html

  db:
    image: mysql:8
    container_name: db_server
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mi_app
      MYSQL_USER: user
      MYSQL_PASSWORD: password
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Ejercicio 6

Se está desarrollando un formulario de registro de usuarios con las siguientes reglas:

- Campo: Edad
  - Debe ser un número entero
  - Rango válido: entre 18 y 65 (ambos incluidos)
- Campo: Nombre de usuario
  - Longitud mínima: 5 caracteres
  - Longitud máxima: 12 caracteres
  - Solo puede contener letras y números
- Campo: Contraseña
- Longitud mínima: 8 caracteres
- Debe contener al menos:
  - 1 letra
  - 1 número

Ejercicios

- Particiones de equivalencia

  Para cada campo:

  - Identificar clases válidas
  - Identificar clases inválidas
- Análisis de valores límite
  - Indicar los valores a probar en los límites para cada campo.

Solución

Campo: Edad (18–65)

ID	Tipo de prueba	Entrada	Resultado esperado	Clase
E1	Valor límite	17	Rechazado	Inválida (límite inferior -1)
E2	Límite inferior	18	Aceptado	Válida
E3	Justo superior	19	Aceptado	Válida
E4	Valor medio	30	Aceptado	Válida
E5	Límite superior	65	Aceptado	Válida
E6	Fuera de rango	66	Rechazado	Inválida
E7	Tipo inválido	"abc"	Rechazado	Inválida (no numérico)
E8	Decimal	25.5	Rechazado	Inválida

Campo: Nombre de usuario (5–12 caracteres alfanuméricos)

ID	Tipo de prueba	Entrada	Resultado esperado	Clase
U1	Valor límite	"abcd"	Rechazado	Inválida (<5)
U2	Límite inferior	"abcde"	Aceptado	Válida
U3	Valor medio	"usuario7"	Aceptado	Válida
U4	Límite superior	"usuario1234"	Aceptado	Válida
U5	Fuera de rango	"usuario12345a"	Rechazado	Inválida (>12)
U6	Caracteres inválidos	"user@12"	Rechazado	Inválida
U7	Solo números	"12345"	Aceptado	Válida

Campo: Contraseña (≥8, letra + número)

ID	Tipo de prueba	Entrada	Resultado esperado	Clase
P1	Valor límite	"abc1234"	Rechazado	Inválida (<8)
P2	Límite inferior	"abc12345"	Aceptado	Válida
P3	Solo letras	"abcdefgh"	Rechazado	Inválida
P4	Solo números	"12345678"	Rechazado	Inválida
P5	Mezcla válida	"abc12345"	Aceptado	Válida
P6	Largo correcto	"a1b2c3d4e5"	Aceptado	Válida

Ejercicio 7

Dada la siguiente clase:

```python
class GestorUsuarios:

    def registrar_usuario(self, nombre, edad, intentos_previos, bloqueado):
        if bloqueado:
            return "Usuario bloqueado"

        if edad < 0:
            return "Edad inválida"

        if edad < 18:
            return "Menor de edad"

        if intentos_previos > 3:
            return "Demasiados intentos"

        if len(nombre) < 5:
            return "Nombre demasiado corto"

        if len(nombre) > 12:
            return "Nombre demasiado largo"

        if " " in nombre:
            return "Nombre inválido"

        return "Usuario registrado"
```

- Identifica:
  - Todas las condiciones (decisiones) del código
  - Los posibles caminos de ejecución
- Diseña un conjunto mínimo de casos de prueba para lograr:
  - Cobertura de sentencias
  - Cobertura de decisiones
- Indica qué línea o rama cubre cada caso.

Solución

→ Decisiones del código

Hay 7 decisiones principales:

1. bloqueado
2. edad < 0
3. edad < 18
4. intentos_previos > 3
5. len(nombre) < 5
6. len(nombre) > 12
7. " " in nombre

→ Casos de prueba mínimos (cobertura completa)

ID	nombre	edad	intentos	bloqueado	Resultado esperado	Cobertura
C1	Ana	25	1	True	Usuario bloqueado	rama bloqueado
C2	Ana	-1	0	False	Edad inválida	edad < 0
C3	Ana	16	0	False	Menor de edad	edad < 18
C4	Ana	25	5	False	Demasiados intentos	intentos > 3
C5	Ana	25	0	False	Nombre demasiado corto	len < 5
C6	NombreLarguísimoExceso	25	0	False	Nombre demasiado largo	len > 12
C7	"Juan Carlos"	25	0	False	Nombre inválido	contiene espacio
C8	"Juan5"	25	0	False	Usuario registrado	caso válido final

→ Cobertura de sentencias

- Se ejecuta cada return al menos una vez:
  - "Usuario bloqueado" → C1
  - "Edad inválida" → C2
  - "Menor de edad" → C3
  - "Demasiados intentos" → C4
  - "Nombre demasiado corto" → C5
  - "Nombre demasiado largo" → C6
  - "Nombre inválido" → C7
  - "Usuario registrado" → C8
- Cobertura de sentencias = 100%

→ Cobertura de decisiones

Cada condición tiene:

- True y False evaluados en el conjunto de pruebas

Decisión	True cubierto	False cubierto
bloqueado	C1	C2–C8
edad < 0	C2	C3–C8
edad < 18	C3	C4–C8
intentos > 3	C4	C5–C8
len < 5	C5	C6–C8
len > 12	C6	C7–C8
contiene espacio	C7	C8

Pistas:

* No basta con “probar todo”, hay que cubrir todas las ramas.
* El orden de los if importa: el flujo se detiene en el primer return.
* Muchos fallais al no llegar al caso válido final.
* Este ejercicio es típico que parece sencillo pero requiere pensar caminos.

Ejercicio 8

Se te proporciona una clase funcional, pero sin documentación. Tu tarea es:

- Añadir JavaDoc completo a la clase y a todos sus métodos.
- Explicar correctamente:
  - qué hace cada método
  - parámetros (@param)
  - valores de retorno (@return)
  - posibles errores o condiciones especiales (@throws si aplica)
- Generar la documentación HTML usando herramienta JavaDoc (simulada o real). Opcional

```java
public class CalculadoraIMC {

    public double calcularIMC(double peso, double altura) {
        if (altura <= 0) {
            throw new IllegalArgumentException("Altura no válida");
        }
        return peso / (altura * altura);
    }

    public String clasificarIMC(double imc) {
        if (imc < 18.5) {
            return "Bajo peso";
        } else if (imc < 25) {
            return "Normal";
        } else if (imc < 30) {
            return "Sobrepeso";
        } else {
            return "Obesidad";
        }
    }
}
```

Solución

```java
/**
 * Clase que proporciona métodos para calcular y clasificar el Índice de Masa Corporal (IMC).
 *
 * El IMC es un valor que relaciona el peso y la altura de una persona para estimar su estado físico.
 *
 * @author DAW
 * @version 1.0
 */
public class CalculadoraIMC {

    /**
     * Calcula el Índice de Masa Corporal (IMC) a partir del peso y la altura.
     *
     * @param peso Peso de la persona en kilogramos.
     * @param altura Altura de la persona en metros. Debe ser mayor que 0.
     * @return Valor numérico del IMC.
     * @throws IllegalArgumentException si la altura es menor o igual a 0.
     */
    public double calcularIMC(double peso, double altura) {
        if (altura <= 0) {
            throw new IllegalArgumentException("Altura no válida");
        }
        return peso / (altura * altura);
    }

     /**
      * Clasifica el valor del IMC en una categoría estándar de salud.
      *
      * @param imc Valor del Índice de Masa Corporal calculado previamente.
      * @return Cadena con la categoría: Bajo peso, Normal, Sobrepeso u Obesidad.
      */
     public String clasificarIMC(double imc) {
         if (imc < 18.5) {
             return "Bajo peso";
         } else if (imc < 25) {
             return "Normal";
         } else if (imc < 30) {
             return "Sobrepeso";
         } else {
             return "Obesidad";
         }
     }
}
```

Ejercicio 10

Se te proporciona una clase que procesa una lista de números.
El programa funciona, pero es **ineficiente**.

Tu tarea es:

1. Identificar problemas de rendimiento.
2. Reescribir el código optimizado.
3. Explicar qué mejoras has aplicado.

```
public class ProcesadorNumeros {

    public List<Integer> obtenerParesMayoresQueDiez(List<Integer> numeros) {

        List<Integer> resultado = new ArrayList<>();

        for (int i = 0; i < numeros.size(); i++) {

            int numero = numeros.get(i);

            if (numero % 2 == 0) {
                if (numero > 10) {
                    if (!resultado.contains(numero)) {
                        resultado.add(numero);
                    }
                }
            }
        }

        return resultado;
    }
}
```

Solución

```java
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.ArrayList;

public class ProcesadorNumeros {

    public List<Integer> obtenerParesMayoresQueDiez(List<Integer> numeros) {

        Set<Integer> resultado = new HashSet<>();

        for (int numero : numeros) {

            if (numero > 10 && numero % 2 == 0) {
                resultado.add(numero);
            }
        }

        return new ArrayList<>(resultado);
    }
}
```

1. Eliminación de for clásico

- Antes: acceso por índice (get(i))
- Ahora: enhanced for loop
- Más legible y ligeramente más eficiente

2. Reducción de complejidad lógica

Antes:

if (a)

  if (b)

    if (c)

Ahora:

      if (numero > 10 && numero % 2 == 0)

Menos ramas → menos coste de evaluación

3. Eliminación de contains() en lista

Antes:

- resultado.contains(numero)

Ahora:

- HashSet

Mejora importante de rendimiento

4. Cambio de estructura de datos

- ArrayList → HashSet
- Evita duplicados automáticamente